

Глава 7: Испытание лабиринтом

Добавляем камеру, сбрасываем груз, выбираемся из тупиков и находим финиш!

В Главе 6 ты построил исследователя лабиринта, который посещает ячейки одну за другой. Но он застревал в тупиках и не мог видеть ничего на полу. Теперь добавим камеру для распознавания цветов, мотор-сбрасыватель для доставки груза и возврат назад для выхода из тупиков.

Что ты изучишь

1. Использование камеры для определения цвета пола
 2. Реакция на цвета (синий = зона сброса, красный = финиш)
 3. Управление мотором-сбрасывателем для выгрузки груза
 4. Почему тупики — это проблема
 5. Возврат назад с помощью стека пути
 6. Создание полного робота для испытания лабиринтом
-

Камера

Камера смотрит на пол и может определять цвета. Она работает не так, как лидар — вместо измерения расстояния она сообщает, какой цвет видит.

Чтение цветов

```
import time
from evabot.components.sensors import Camera

camera = Camera()
camera.start()

time.sleep(2)

# Получаем цвет как HSV (Оттенок, Насыщенность, Яркость)
hsv = camera.get_color()
if hsv:
    h, s, v = hsv
    print(f"Оттенок: {h}, Насыщенность: {s}, Яркость: {v}")

camera.stop()
```

HSV — это способ описания цвета: - **Оттенок (Hue)** (0-179): Сам цвет (красный=0, жёлтый=30, зелёный=60, синий=120) - **Насыщенность (Saturation)** (0-255): Насколько яркий цвет (0=серый, 255=чистый цвет) - **Яркость (Value)** (0-255): Насколько светло (0=темно, 255=ярко)

Сопоставление с именованными цветами

Вместо чтения сырых HSV можно спросить «это синий?»

```
score = camera.match_color("blue")
print(f"Уверенность в синем: {score:.2f}")
```

`match_color()` возвращает оценку уверенности от 0.0 до 1.0: - **0.0** = точно не этот цвет - **0.05+** = вероятно этот цвет - **1.0** = всё поле зрения этого цвета

Доступные цвета: "red", "yellow", "green", "blue", "white", "black"

Сканер цветов

```
import time
from evabot.components.sensors import Camera

camera = Camera()
camera.start()

time.sleep(2)

COLORS = ["red", "yellow", "green", "blue", "white", "black"]

# Сканируем 10 раз
for i in range(10):
    scores = {c: camera.match_color(c) for c in COLORS}
    best = max(scores, key=scores.get)
    conf = scores[best]

    if conf > 0.05:
        print(f"Вижу: {best} (уверенность: {conf:.2f})")
    else:
        print("Нет чёткого цвета")

    time.sleep(0.5)

camera.stop()
```

Попробуй: Положи цветную бумагу под робота и запусти код. Подкладывая разные цвета, чтобы увидеть, как он их определяет.

Упражнение 7.1: Цветовая тревога

Выводи «ТРЕВОГА!», когда камера видит красный. Выводи «ОК» для любого другого цвета.

Решение

```
import time
from evabot.components.sensors import Camera

camera = Camera()
camera.start()

time.sleep(2)
```

```
for i in range(20):
    if camera.match_color("red") > 0.05:
        print("ТРЕВОГА! Обнаружен красный!")
    else:
        print("OK")
        time.sleep(0.5)

camera.stop()
```

Мотор-сбрасыватель

Сбрасыватель — это обычный мотор (Servo42D) с CAN ID 5. Вращение открывает механизм, который выпускает груз (например, мячик или маленький предмет).

Простой сброс

```
import time
from evabot import Servo42D

dropper = Servo42D(5)
dropper.start()

# Вращаем для сброса груза
dropper.run(30)      # 30 об/мин
time.sleep(1.0)      # 1 секунду
dropper.run(0)       # стоп

dropper.stop()
```

Вот и всё! Включаем мотор, ждём, останавливаем. Механическая конструкция делает остальное.

Упражнение 7.2: Сброс на синем

Совмести камеру и сбрасыватель: при обнаружении синего — сбрось груз.

Решение

```
import time
from evabot import Servo42D
from evabot.components.sensors import Camera

camera = Camera()
camera.start()

dropper = Servo42D(5)
dropper.start()

time.sleep(2)
```

```

print("Жду синий...")
while True:
    if camera.match_color("blue") > 0.05:
        print("Синий обнаружен! Сбрасываю груз...")
        dropper.run(30)
        time.sleep(1.0)
        dropper.run(0)
        print("Сброшено!")
        break
    time.sleep(0.3)

dropper.stop()
camera.stop()

```

Добавляем камеру в исследователь лабиринта

Теперь добавим определение цвета в наш исследователь из Главы 6. После каждого перемещения робот проверяет пол:

- **Синий** = зона сброса → выгрузить груз
- **Красный** = финишная линия → прекратить исследование

```

import time
from evabot import Robot, MecanumDrive, RPLidarC1, Servo42D
from evabot.components.sensors import Camera

robot = Robot()
robot.drive = MecanumDrive()
robot.lidar = RPLidarC1()
robot.start()

camera = Camera()
camera.start()

dropper = Servo42D(5)
dropper.start()

payload_dropped = False

time.sleep(3)

# ... после каждого move_to_wall() в цикле исследования:

# Проверяем финиш
if camera.match_color("red") > 0.05:
    print("КРАСНАЯ ЗОНА – ФИНИШ!")
    break

# Проверяем зону сброса
if not payload_dropped:
    if camera.match_color("blue") > 0.05:
        print("СИНЯЯ ЗОНА! Сбрасываю груз...")

```

```

dropper.run(30)
time.sleep(1.0)
dropper.run(0)
payload_dropped = True

```

Важные детали: - Сначала проверяем красный (финиш важнее сброса) - Флаг `payload_dropped` гарантирует, что сбрасываем только один раз - Проверки выполняются после каждого перемещения, когда робот стоит

Проблема тупиков

У исследователя из Главы 6 есть большая проблема. Посмотри на этот лабиринт:

```

+---+---+---+---+
|           |
+   +---+---+   +
|   |           |   |
+   +   +   +   +
|           |   |   |
+---+---+---+---+

```

Робот начинает слева внизу и едет влево, влево, влево... упирается в конец. Все соседи посещены. **Он останавливается, хотя правая часть не исследована!**

Шаг 1: (0,0) → влево

Шаг 2: (0,1) → влево

Шаг 3: (0,2) → влево

Шаг 4: (0,3) → ЗАСТРЯЛ! Все соседи посещены.
Но (1,0), (2,0) и т.д. ещё не исследованы!

Возврат назад со стеком пути

Решение простое: **запоминай путь, которым шёл, и иди по нему назад, когда застрял.**

Как это работает

Представь, что ты оставляешь хлебные крошки на ходу:

Вперёд: (0,0) →лево→ (0,1) →лево→ (0,2) →лево→ (0,3) → тупик!
[push] [push] [push]

Назад: (0,3) →право→ (0,2) → есть неисследованные? Нет.
[pop]
(0,2) →право→ (0,1) → есть неисследованные? Нет.
[pop]
(0,1) →право→ (0,0) → есть неисследованные? ДА!
[pop]

Дальше: (0,0) →вперёд→ (1,0) → новая территория!

[push]

Стек пути

Стек — это список, в который всегда добавляют и убирают с конца (как стопка тарелок — последняя положенная снимается первой).

```
path = []
```

```
# Идём вперёд: записываем, откуда пришли
path.append((cell_x, cell_y, direction))
```

```
# Идём назад: достаём и разворачиваем направление
prev_x, prev_y, came_from = path.pop()
back_direction = OPPOSITE[came_from]
```

Каждая запись в стеке хранит: - `cell_x, cell_y` — ячейка, в которой мы были до перемещения - `direction` — куда мы поехали

Для возврата достаём запись и движемся в **противоположном** направлении.

Упражнение 7.3: Разбираемся в стеке

Для данного лабиринта и пути, как выглядит стек после 4 ходов?

```
+---+---+---+
|       |   |
+---+   +   +
|   |       |
+---+---+---+
```

Старт в (0,0), приоритет: лево, назад, право, вперёд

Решение

Шаг 1: В (0,0), идём вперёд в (1,0) → стек: [(0,0,вперёд)]

Шаг 2: В (1,0), идём влево в (1,1) → стек: [(0,0,вперёд), (1,0,влево)]

Шаг 3: В (1,1), идём вперёд в (2,1) → стек: [(0,0,вперёд), (1,0,влево), (1,1,вперёд)]

Шаг 4: В (2,1), идём вправо в (2,0) → стек: [(0,0,вперёд), (1,0,влево), (1,1,вперёд), (2,1,вправо)]

Если застряли в (2,0), достаём (2,1,вправо) → идём влево (противоположно) назад в (2,1).

Цикл с возвратом

Вот как меняется цикл исследования:

```
path = [] # стек для возврата
```

```
for step in range(200):
```

```
    # Сканируем и обновляем карту (как раньше)
```

```

visited.add((cell_x, cell_y))
# ... сканируем стены, записываем ...

# Ищем непосещённого соседа
neighbors = get_unvisited_neighbors(cell_x, cell_y)

if neighbors:
    # Вперёд: едем в новую ячейку, добавляем в стек
    direction, nx, ny = neighbors[0]
    path.append((cell_x, cell_y, direction))
    robot.move_to_wall(direction)
    cell_x, cell_y = nx, ny

elif path:
    # Тупик: возвращаемся, пока не найдём неисследованный путь
    print("Тупик, возвращаюсь...")
    while path:
        prev_x, prev_y, came_from = path.pop()
        back_dir = OPPOSITE[came_from]
        robot.move_to_wall(back_dir)
        cell_x, cell_y = prev_x, prev_y

        if get_unvisited_neighbors(cell_x, cell_y):
            break # нашли ячейку с неисследованными соседями!
    else:
        break # вернулись к старту, лабиринт полностью исследован

    continue # возвращаемся к началу цикла для сканирования

else:
    break # нет соседей, нет пути – готово

```

Три случая: 1. **Есть непосещённый сосед** → едем туда (push в стек) 2. **Нет соседа, но стек не пуст** → возвращаемся (pop из стека) 3. **Нет соседа, стек пуст** → полностью исследовано

Полное испытание лабиринтом

Собираем всё вместе: камера, сбрасыватель, возврат назад и исследователь.

```

import time
from evabot import Robot, MecanumDrive, RPLidarC1, Servo42D
from evabot.components.sensors import Camera

# Настройка
robot = Robot()
robot.drive = MecanumDrive()
robot.lidar = RPLidarC1(max_range=1.0)
robot.start()

camera = Camera()
camera.start()

```

```

dropper = Servo42D(5)
dropper.start()

time.sleep(3)
print("Нажми ENTER для старта!")
input()

# Вспомогательные словари
DIR_DELTA = {0: (1, 0), 90: (0, -1), 180: (-1, 0), 270: (0, 1)}
OPPOSITE = {0: 180, 180: 0, 90: 270, 270: 90}
DIR_NAMES = {0: "Вперёд", 90: "Вправо", 180: "Назад", 270: "Влево"}
WALL_THRESHOLD = 0.25

# Карта
visited = set()
walls = {}
path = []
payload_dropped = False

def set_wall(x, y, d, has_wall):
    walls[(x, y, d)] = has_wall
    dx, dy = DIR_DELTA[d]
    walls[(x + dx, y + dy, OPPOSITE[d])] = has_wall

# Старт
x, y = 0, 0

for step in range(200):
    print(f"\n--- Ячейка ({x},{y}) Шаг {step+1} ---")

    # Сканируем стены
    visited.add((x, y))
    for angle in [0, 90, 180, 270]:
        dist, __, q = robot.lidar.check_wall(angle)
        has_wall = dist is not None and q is not None and q > 0.3 and dist < WALL_THRESHOLD
        set_wall(x, y, angle, has_wall)

    # Ищем непосещённого соседа (приоритет: лево, назад, право, вперёд)
    neighbors = []
    for d in [270, 180, 90, 0]:
        dx, dy = DIR_DELTA[d]
        nx, ny = x + dx, y + dy
        if walls.get((x, y, d)) is False and (nx, ny) not in visited:
            neighbors.append((d, nx, ny))

    if neighbors:
        direction, nx, ny = neighbors[0]
        print(f" Двигаюсь {DIR_NAMES[direction]} -> ({nx},{ny})")
        path.append((x, y, direction))
        robot.move_to_wall(direction)
        x, y = nx, ny

    elif path:
        print(" Возвращаюсь...")

```



```

while path:
    px, py, came_from = path.pop()
    robot.move_to_wall(OPPOSITE[came_from])
    x, y = px, py
    # Есть ли неисследованные соседи?
    for d in [270, 180, 90, 0]:
        dx, dy = DIR_DELTA[d]
        nx, ny = x + dx, y + dy
        if walls.get((x, y, d)) is False and (nx, ny) not in visited:
            print(f" Нашёл путь в ({x},{y})")
            break
    else:
        continue # нет соседей, продолжаем возврат
    break # нашли соседей, останавливаем возврат
else:
    print(" Полностью исследовано!")
    break
continue

else:
    print(" Полностью исследовано!")
    break

# Проверяем цвета
if camera.match_color("red") > 0.05:
    print(" КРАСНЫЙ – ФИНИШ!")
    break

if not payload_dropped and camera.match_color("blue") > 0.05:
    print(" СИНИЙ – Сбрасываю груз!")
    dropper.run(30)
    time.sleep(1.0)
    dropper.run(0)
    payload_dropped = True

print(f"\nПосещено {len(visited)} ячеек!")
robot.drive.halt()
dropper.stop()
camera.stop()
robot.stop()

```

Упражнение 7.4: Подсчёт возвратов

Добавь счётчик, который отслеживает, сколько раз робот возвращался назад. Выведи его в конце.

Решение

```

# Добавь перед циклом:
backtrack_count = 0

```

```

# В секции возврата, после "Возвращаюсь...":
backtrack_count += 1

```

```
# В конце:  
print(f"Возвратов: {backtrack_count}")
```

Упражнение 7.5: Ускорение при возврате

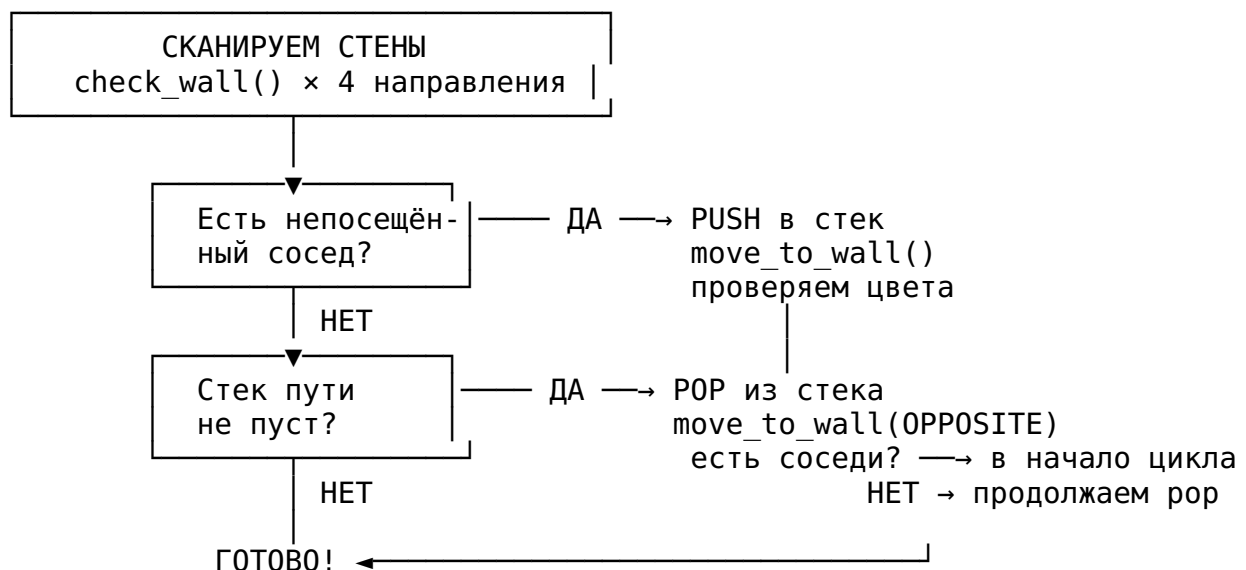
При возврате робот уже знает, что путь безопасен. Сделай его быстрее при возврате ($speed=0.5$) и медленнее при исследовании ($speed=0.3$).

Решение

```
# Движение вперёд:  
robot.move_to_wall(direction, speed=0.3)  
  
# Возврат:  
robot.move_to_wall(OPPOSITE[came_from], speed=0.5)
```

Робот уже посетил эти ячейки, поэтому знает, что сюрпризов не будет. Быстрый возврат означает меньше потраченного времени в тупиках!

Как всё работает вместе



Робот всегда исследует все достижимые ячейки. Он может посетить некоторые ячейки дважды (при возврате), но никогда не пропустит ячейку и не заикнется.

Итоги

Ты научился:

- Читать цвет пола с помощью `camera.get_color()` и `camera.match_color()`

- Реагировать на определённые цвета (синий = сброс, красный = финиш)
- Управлять мотором-сбрасывателем через Servo42D (5) . run (speed)
- Почему тупики ломают простое исследование
- Возврат назад со стеком пути (DFS)
- Push при движении вперёд, pop когда застрял
- Полное испытание лабиринтом: исследование, определение цветов, доставка груза, поиск финиша

Робот для испытания лабиринтом в одном предложении:

Сканируй стены, выбери непосещённого соседа, поезжай туда, проверь цвета, а когда застрял — возвращайся, пока не найдёшь новый путь.

Что дальше?

Поздравляем! Ты построил робота, который умеет: - Точно ездить на механуме - Видеть стены лидаром - Различать цвета камерой - Проходить лабиринт от старта до финиша - Доставлять груз в нужное место

Идеи для дальнейшего изучения: - **Кратчайший путь** — После исследования найти самый быстрый маршрут от старта до финиша - **Оптимизация скорости** — Как быстро можно пройти лабиринт? - **Несколько грузов** — Сбрасывать разные предметы в зонах разного цвета - **Карта лабиринта** — Показать полную карту после исследования